Matchmaking compartilhado entre servidores

Analise de viabilidade do bot CAPS Discord

Gerado em 10/10/2026 a partir da investigacao do codigo. Sem modificacoes de codigo, sem envios ao Discord.

Conclusao de viabilidade

Viavel, mas nao existe hoje. Requer mudancas pequenas-medias, nao so configuracao.

Ja existe

- Fila global unica em services/dataService.js (loadQueue/saveQueue) + database/queue.json ({lobbies, waitingLists}).
- Identidade global por discordId (utils/lobbyUtils.js: getPlayerStatsKey, getStoredPlayerStats, upsertPlayerStats).
- Anti-duplicata global em fila (application/use-cases/enterQueue.js:123 via findLobbyByPlayer).
- Balanceamento, criacao de canais de voz e movimentacao (startMatch.js: startMatch; lobbyUtils.js: createLobbyChannels, createTeamChannelsForLobby, movePlayersToTeamChannels).
- Lock em memoria por operacao (services/dataService.js: withQueueOperationLock).
- Gate simples de guild (utils/lobbyUtils.js: isGuildAllowed; config.json: allowedGuildIds).
- Envio de DM de texto simples (services/tempVoiceService.js: handleInviteCommand).

Nao existe

- Isolamento de fila por servidor. Nem fila compartilhada opt-in.
- Setting por servidor persistido no backend.
- guildId / originGuildId em lobby, waitingLists, playerStats, currentMatch.
- Criacao ou recuperacao de invites do Discord (createInvite, fetchInvites, discord.gg).
- Uso de webhooks. grep webhook|Webhook retorna 0. Tudo e channel.send direto.
- Config por guild. config.json tem IDs escalares globais (voiceChannels, textChannels, roles, tempVoice.hubChannelId). setup-servidor.js sobrescreve o mesmo arquivo global.

A premissa "gerencia via webhooks" esta incorreta no codigo atual. O fluxo e discord.js direto com GatewayIntentBits.Guilds, GuildMessages, GuildVoiceStates, MessageContent, GuildMembers em index.js:62-70.

1. As filas sao isoladas por servidor? Onde?

Nao. Sao globais e sem particao.

- services/dataService.js:104,173-183: doc unico _id: queue com {lobbies:{}, waitingLists:{}}. loadQueue/saveQueue, loadCurrentMatch, loadPlayerStats, loadSystemMeta nao aceitam guildId.
- utils/lobbyUtils.js:508 getOpenLobby, 522 findLobbyByPlayer, 570 findLobbyBySelector, 515 findLobbyByChannelId: filtram so por mode/format/letter/status, nunca guildId.
- application/use-cases/enterQueue.js:218-229: lobby = {id, mode, format, tier, letter, waitingChannelId, parentId, requiredPlayers, players[], status} - sem guildId/originGuildId. id = mode-format-tier-letter colidiria entre guilds.
- enterQueue.js:48 getWaitingListKey: classic:5x5, classic:5x5:s, aram:2x2 - sem guild.
- database/queue.json, currentMatch.json, playerStats.json: nenhum campo guildId em lobby/player/match.
- Unico escopo por guild hoje e efemero: guild passado para criar canais/anunciar (queueCommandHandlers.js:34-36, matchCommandHandlers.js:28-31, startMatch.js:1-6), chave de lock ${guildId}:global:queue, e systemMeta.recentVictories[].guildId (legacyCommands.js:256,333,1535).
- Isolamento real hoje e por processo+DB: MONGODB_DB=caps-bot-teste (dataService.js:8) + allowedGuildIds=["1558217291123138561"] (config.json:54). Nao ha multi-guild no mesmo processo.

Consequencia: dois servidores no mesmo bot/DB hoje disputariam o mesmo classic-5x5-a.

2. Encaixe minimo na arquitetura atual

A arquitetura (docs/ARCHITECTURE.md, PROJECT_CONTEXT.md) ja separa domain/queue/selection.js, application/use-cases/enterQueue.js/startMatch.js, services/dataService.js e commands/handlers. O ponto de menor atrito:

- 1. Adicionar guildSettings persistido ({guildId: {sharedMatchmaking: bool, inviteChannelId, ...}}).

- 2. Marcar cada entrada de fila com originGuildId + originChannelId, e cada lobby/match com scope shared|local, destinationGuildId e playerOrigins.
- 3. Bifurcar enterQueue: se guild com shared=false, caminho atual inalterado; se true, pool compartilhado por mode:format:tier.
- 4. Resolver destino so no startMatch, criar canais so na guild destino, notificar ambas as guilds + DM com invite.
- 5. Trocar lock por-guild por lock global para o pool compartilhado.

Nao precisa virar multi-API agora. Monolito + Mongo comporta isso.

3. Pool unico de opt-in vs grupos explicitos

Pool unico (todos com shared=true na mesma fila por mode:format)

- Pro: mudanca minima. Reusa getWaitingListKey + getOpenLobby. Sem UI de grupos. Ideal para 2-3 servidores parceiros.
- Contra: mistura comunidades sem controle, ranking/MMR divergente contamina balanceamento, moderacao cruzada, escolha de destino mais sensivel.

Grupos explicitos (sharedGroupId: [guildA, guildB])

- Pro: controle de quem joga com quem, temporadas/rankings por grupo, destino confinado, expansao segura.
- Contra: necessita CRUD de grupos, comando admin, migracao, testes extras.

Recomendacao

Fase 1: pool unico opt-in (sharedMatchmaking: boolean por guild). E o menor incremento que atende "2+ servidores" e valida invites/DMs/concorrencia. Fase 2: sharedGroupId opcional quando houver 3+ guilds ou exigencia de isolamento. Modelar guildSettings ja com campo sharedGroupId=null para nao migrar depois.

4. Identidade, duplicatas e multi-match

- Identidade cross-guild funciona: discordId (author.id) e globalmente unico no Discord. playerStats chaveado por puuid || discord:<id> (lobbyUtils.js:1229) ja colapsa o mesmo usuario. Nick#TAG e so display (duplicate_nickname em enterQueue.js:196-202).
- findLobbyByPlayer ja impede dupla entrada em lobbies, mas so varre queueData.lobbies, nao currentMatchData.matches nem waitingLists de outra guild. Para compartilhado, verificar nos 3 lugares: lobbies[].players + waitingLists[*] + matches[*].match.{teamOne,teamTwo} por discordId.
- Evitar multi-match: no startMatch, revalidar dentro do lock global que nenhum dos 10 discordIds esta em matches[*].active. Hoje so checa matches[lobby.id]?.active (startMatch.js:24).
- Locks atuais sao ${guildId}:global:queue, logo guildA e guildB entrariam em paralelo no mesmo doc. Exige lock unico para pool compartilhado, ex. shared:global:queue ou shared:<mode>:<format>. Limitacao pre-existente: lock e Map em memoria (dataService.js:151-171) - seguro em 1 instancia, inseguro em multi-instancia.

5. Concorrencia, cancelamento, timeout, mudanca de preferencia

- Concorrencia: centralizar loadQueue+loadCurrentMatch -> mutate -> save sob lock global compartilhado. handleVoteCommandFlow (matchCommandHandlers.js:118) ja mostra o padrao de lock para evitar read-modify-write perdido.
- Cancelamento/!sair: handleLeaveCommand, handleRemoveCommand e handleCleanupRoomsCommand operam por message.guild. No compartilhado, !sair deve remover o discordId do lobby compartilhado de qualquer guild de origem, e updateQueueDashboard deve atualizar todas as guilds participantes (hoje edita systemMeta.lastQueueMessageId global - lobbyUtils.js:1935-1959 - que colidiria com 2 guilds; precisa virar por guild).
- Timeouts: pendingAutoStarts: Map(lobbyId->timeout) e em memoria e por lobbyId. Funciona para shared se triggerAutoStart guardar tambem destinationGuild e anunciar nas duas guilds. Restart perde o timer - index.js:440-450 ja retoma lobbies cheios; estender para re-resolver destino.
- Mudanca de preferencia no meio da fila: regra de produto pendente. Sugestao tecnica: shared=false nao dissolve lobby compartilhado ativo; apenas direciona novas entradas para pool local. Comando staff !shared leave remove entradas pendentes daquela guild do pool shared.

6. Escolha do destino e preservacao de origem

Preservar por jogador: playerEntry += { originGuildId, originChannelId, originGuildName }; lobby/match += { scope, destinationGuildId, destinationChannelIds, playerOrigins }.

Criterios de destino (ordem sugerida, deterministica para testes): 1) Maioria de origem (ex. 6 de A + 4 de B vai para A, menos invites). 2) Desempate: guild que completou a fila (ultimo !entrar), ou round-robin por lobbyId, ou menor carga (matches ativos). 3) Fallback configuravel: preferredHostGuildId por grupo.

Notificacoes/estatisticas: anunciar sendMatchStartAnnouncement + postMatchHistoryLog na destino e resumo com invite na origem. Estatisticas (playerStats.modes) hoje sao globais sem guildId - manter global na Fase 1, mas carimbar match.originGuilds[] e destinationGuildId no historico.

7. O bot cria/recupera invites? Permissoes e entrega

Nao. Verificado em codigo. Nada a reusar sem criar.

- grep createInvite|fetchInvites|getInvite|discord.gg|CreateInstantInvite|INVITE: 0 hits relevantes. grep webhook: 0.
- Unico "convidar" e services/tempVoiceService.js:426 handleInviteCommand (!convidar @...): faz permissionOverwrites.edit + target.send(DM texto) - sem Invite, sem link, so intra-guild.

Para implementar: bot membro de todas as guilds; permissao CreateInstantInvite na guild destino + canal de destino; intents atuais bastam (Guilds, GuildMembers); config por guild com inviteChannelId; invite com maxAge curto (ex. 3600s), maxUses = teamSize, unique por match. Entrega: DM via user.send() + mencao no canal de origem + embed com link. DM pode falhar (privacidade) - tratar como dmFailed[]. Requisito: so gerar/enviar invite no startMatch (match ready), nunca no enterQueue.

8. Falhas de invite/DM

- Sem permissao / canal invalido: nao deixar lobby em estado meio-criado sem rollback (deleteManagedChannelsForLobby). Avisar origem; partida pode ocorrer so com nativos da destino.
- Invite expirado/invalidado: gerar novo sob lock, persistir match.invite.{code,url,expiresAt}, reenviar so aos pendentes.
- DM fechada: fallback para mencao publica no canal de origem + instrucoes discord.gg/xxx. Registrar inviteDelivery: {sent, dmFailed} por discordId.
- Jogador nao entra na destino: movePlayersToTeamChannels ja tolera notInVoice (lobbyUtils.js:2091-2111). Estender para notInDestinationGuild. Se abaixo do minimo apos X min, staff pode !cancelarstart (restaura lobby via cancelActiveMatch.js: buildLobbyFromMatch).

9. Webhook existente serve?

Nao, porque nao ha webhook. Todo envio e direto: lobbyUtils.js updateQueueDashboard, sendMatchStartAnnouncement, postMatchHistoryLog, postMvpAnnouncement, postDailyRankUpdates, movePlayersToTeamChannels; index.js guildMemberAdd; commands/handlers/*; tempVoiceService.js.

Capacidades adicionais necessarias (discord.js padrao, sem webhook): GuildChannel.createInvite(), Client.guilds.fetch(guildId), Guild.members.fetch(discordId), User.send(). Opcional: canal de anuncios cross-post, botoes com link (ButtonStyle.Link).

Fluxo end-to-end proposto

- 1. Admin A/B ativa: !shared on -> guildSettings[A].sharedMatchmaking=true (Mongo data.guildSettings + fallback database/guildSettings.json).
- 2. Jogador em A entra em voz + !entrar Nick#TAG -> handleEnterCommandFlow valida voz local, chama enterQueue com originGuildId=A.
- 3. enterQueue sob lock global: rejeita se discordId em qualquer lobby/waiting/match ativo; se A shared, busca/cria lobby scope:shared no pool shared:<mode>:<format>; anexa originGuildId.
- 4. Repete em B. Dashboard atualizado em A e B (por-guild lastQueueMessageId).
- 5. Lobby atinge requiredPlayers -> triggerAutoStart (5s, cancelavel via !cancelarstart).
- 6. startMatch({destinationGuild}): elege destino (maioria), cria canais na destino, balanceia (createBalancedTeams), persiste matches[lobbyId], deleta lobby shared.

- 7. So agora: destChannel.createInvite({maxAge,maxUses}) -> persiste match.invite.
- 8. Move quem esta na destino; para cada jogador de outra origem: DM com invite + mencao no canal origem.
- 9. Anuncia embed de times na destino + resumo na origem.
- 10. !votar/!vitoria atualiza playerStats global; postMatchHistoryLog na destino com originGuilds carimbados.

Guild com shared=false usa caminho atual inalterado (scope:local, lock por guild, sem invite).

Menor plano pratico

Settings

- Novo doc guildSettings: {[guildId]: {sharedMatchmaking: bool, sharedGroupId: null, inviteChannelId: null, announceChannelId: null}} + loadGuildSettings/saveGuildSettings em services/dataService.js.
- Comandos staff: !shared on|off|status (+ slash /shared). Guard ManageGuild/Administrator.

Modelo de dados

- queue.lobbies[].{scope, originGuilds[], destinationGuildId?}; lobby.players[].{originGuildId, originChannelId?}; waitingLists chave shared:<mode>:<format>; currentMatch.matches[].match.{scope, destinationGuildId, playerOrigins, invite?}; systemMeta.lastQueueMessageId -> mapa por guild.
- Migracao: defaults scope:local, shared=false = comportamento atual.

Componentes afetados

- services/dataService.js (novas loads/saves, lock global shared:global:queue).
- application/use-cases/enterQueue.js, startMatch.js, cancelActiveMatch.js, registerVictory.js/castVictoryVote.js.
- utils/lobbyUtils.js (getOpenLobby, findLobbyByPlayer, findLobbyBySelector, createLobbyChannels, createTeamChannelsForLobby, movePlayersToTeamChannels, updateQueueDashboard, sendMatchStartAnnouncement, novo createMatchInvite e notifyCrossGuildPlayers).
- commands/handlers/queueCommandHandlers.js, matchCommandHandlers.js, legacyCommands.js (triggerAutoStart, handleLeaveCommand, handleCleanupRoomsCommand, handleResetCommand).
- config.json como fallback + setup-servidor.js passando a escrever em guildSettings.
- index.js: registerSlashCommands, guildCreate, messageCreate, interactionCreate.

Testes (npm test, node --test tests/*.test.js)

- shared-settings.test.js: on/off persiste, off usa pool local.
- shared-queue.test.js: 2 guilds enchem shared:classic:5x5, 1 lobby, originGuildId preservado, duplicata cross-guild rejeitada.
- destination.test.js: maioria vence, empate deterministico, fallback sem maioria.
- invite.test.js (mock createInvite, user.send): invite so no start, nunca no enter; DM fechada -> fallback; sem permissao -> erro tipado sem lobby orfao.
- concurrency.test.js: enters paralelos A+B sob lock global nao duplicam nem estouram requiredPlayers.
- dashboard.test.js: updateQueueDashboard por guild nao sobrescreve lastQueueMessageId da outra.
- Regressao: guild-filter.test.js, tiers-roulette.test.js, cancel-start.test.js, cleanup-rooms.test.js, register-victory.test.js.

Pendencias de produto que bloqueiam o design final

- 1. Ranking global ou por servidor/grupo? Hoje playerStats.modes e global.
- 2. Exigencia de voz na origem para fila shared? Mover voz cross-guild e impossivel.
- 3. Quem pode ativar shared? Ha consentimento mutuo entre A e B?
- 4. Destino fixo (arena principal) vs dinamico? Fixo simplifica tudo.
- 5. Se invite falhar: cancela partida ou joga desfalcado?
- 6. Limite de grupos: 2 fixos ou N? Convites permanentes ou por partida?
- 7. Moderacao cross-guild: ban em A vale em shared? Quem pode !remover jogador da outra guild?
- 8. Temporada/historico por guild ou consolidado? seasonMeta/seasonHistory hoje globais.

Sem essas definicoes, defaults recomendados: ranking global inalterado, voz exigida na origem, destino por maioria,

falha de invite nao cancela (avisa + joga com presentes), grupo unico implicito.

Arquivos e funcoes de referencia

- services/dataService.js: withQueueOperationLock, loadQueue/saveQueue, loadCurrentMatch, loadPlayerStats, loadSystemMeta
- application/use-cases/enterQueue.js: enterQueue, getWaitingListKey, getPendingRouletteEntries
- application/use-cases/startMatch.js: startMatch
- application/use-cases/cancelActiveMatch.js: buildLobbyFromMatch (via lobbyUtils)
- utils/lobbyUtils.js: isGuildAllowed, getOpenLobby, findLobbyByPlayer, createLobbyChannels, createTeamChannelsForLobby, movePlayersToTeamChannels, updateQueueDashboard, sendMatchStartAnnouncement, postMatchHistoryLog
- commands/handlers/queueCommandHandlers.js: handleEnterCommandFlow
- commands/handlers/matchCommandHandlers.js: handleStartCommandFlow, handleVoteCommandFlow
- commands/legacyCommands.js: triggerAutoStart, getRecentVictoryForGuild, handleLeaveCommand, handleCleanupRoomsCommand
- services/tempVoiceService.js: handleInviteCommand (DM sem Invite)
- index.js: registerSlashCommands, messageCreate, interactionCreate, guildCreate, voiceStateUpdate
- config.json: allowedGuildIds, voiceChannels, textChannels, tempVoice
- setup-servidor.js | admin/app.js + admin/service.js | database/*.json | tests/*.test.js